

SQL JOINS

Visual Guide - Row by Row

See how SQL reads two tables, row by row, and builds the result.

Color legend:

Color	Meaning
Yellow row	SQL is reading this row now
Green row	Match found between tables
Red row	No match - row is removed (INNER JOIN)

Our two tables:

EMPLOYEES (LEFT table, from FROM clause):

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

DEPARTMENTS (RIGHT table, from JOIN clause):

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

The bridge: dept_id is in both tables. SQL uses this column to match rows.

Part 1: What Does a JOIN Do?

You have the EMPLOYEES table. It has first_name and dept_id. But it does NOT have the department name. The department name is in the DEPARTMENTS table. A JOIN connects these two tables using the dept_id column. dept_id is the bridge between them.

When SQL runs a JOIN, it does this step by step:

1. Read one row from the LEFT table (employees)
2. Look at the dept_id value in that row
3. Search the RIGHT table (departments) for the same dept_id value
4. If found: read the SELECT columns from BOTH tables and show them on screen
5. If not found: the row is removed (INNER JOIN) or kept with NULL (LEFT JOIN)

Now let us see this happen, step by step, with our two tables.

Part 2: INNER JOIN - Row by Row

INNER JOIN = keep only rows that match in BOTH tables. No match = row is removed from the result.

```
SELECT e.first_name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

How to read: FROM employees = LEFT table. INNER JOIN departments = RIGHT table. ON e.dept_id = d.dept_id = the matching rule (the bridge). SELECT e.first_name, d.dept_name = what columns to show from each table.

Step 1: SQL reads the first row from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

dept_id = 90 ---> found "Executive" in departments OK

SELECT reads from both tables: e.first_name = "Steven" + d.dept_name = "Executive"

Screen shows: Steven | Executive

Step 2: SQL reads the next row from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

dept_id = 90 ---> found "Executive" in departments OK

SELECT reads from both tables: e.first_name = "Neena" + d.dept_name = "Executive"

Screen shows: Neena | Executive

Step 3: SQL reads the next row from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

dept_id = NULL ---> NULL can not match anything NO MATCH

Lex is REMOVED from result! (NULL means "no value" - SQL cannot compare it with any number)

Step 4: SQL reads the next row from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

dept_id = 10 ---> found "Administration" in departments OK

SELECT reads from both tables: e.first_name = "Diana" + d.dept_name = "Administration"

Screen shows: Diana | Administration

Step 5: SQL reads the last row from employees

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

dept_id = 10 ---> found "Administration" in departments OK

SELECT reads from both tables: e.first_name = "Kimberely" + d.dept_name = "Administration"

Screen shows: Kimberely | Administration

What about departments with no employees?

Marketing (dept_id = 20) and Accounting (dept_id = 110) have no employee with that dept_id. SQL does NOT find a match. So they are removed from the result too.

INNER JOIN Final Result:

first_name	dept_name
------------	-----------

Steven	Executive
Neena	Executive
Diana	Administration
Kimberely	Administration

We started with 5 employees + 4 departments. Result has 4 rows.

Lex is gone (NULL dept_id). Marketing is gone (no employee). Accounting is gone (no employee).

Only matching rows survived.

Part 3: LEFT JOIN - What is Different?

LEFT JOIN = keep ALL rows from the LEFT table (employees). If a row has no match, write NULL. Do NOT remove any left table row.

```
SELECT e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

Steps 1, 2, 4, 5 are the same as INNER JOIN (they all match). The only difference is Step 3 (Lex).

Step 3 with LEFT JOIN (Lex):

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

dept_id = NULL ----> NO MATCH

But LEFT JOIN does NOT remove Lex! He stays in the result.

SELECT reads: e.first_name = "Lex" + d.dept_name = NULL (no match in departments)

Screen shows: Lex | NULL

LEFT JOIN Final Result:

first_name	dept_name
Steven	Executive
Neena	Executive
Lex	NULL
Diana	Administration
Kimberely	Administration

5 rows now! Lex is back with NULL. Marketing and Accounting are still gone (they are in the RIGHT table). LEFT JOIN protects only the LEFT table.

Part 4: RIGHT JOIN - What is Different?

RIGHT JOIN = keep ALL rows from the RIGHT table (departments). If a row has no match, write NULL. Do NOT remove any right table row.

```
SELECT e.first_name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

Steven, Neena, Diana, Kimberly all match (same as INNER JOIN). The difference: Marketing and Accounting are now kept. But Lex is removed (he is in the LEFT table, and RIGHT JOIN protects the RIGHT table).

Marketing (dept_id = 20) with RIGHT JOIN:

emp_id	first_name	dept_id
100	Steven	90
101	Neena	90
102	Lex	NULL
107	Diana	10
178	Kimberely	10

dept_id	dept_name
10	Administration
20	Marketing
90	Executive
110	Accounting

dept_id = 20 ---> no employee has dept_id = 20 NO MATCH

But RIGHT JOIN does NOT remove Marketing! It stays.

SELECT reads: e.first_name = NULL + d.dept_name = "Marketing"

Screen shows: NULL | Marketing

(The same happens for Accounting, dept_id = 110)

RIGHT JOIN Final Result:

first_name	dept_name
Diana	Administration
Kimberely	Administration
NULL	Marketing
Steven	Executive
Neena	Executive
NULL	Accounting

6 rows! Marketing and Accounting are back with NULL. But Lex is gone (he is in the LEFT table). RIGHT JOIN protects only the RIGHT table.

Part 5: FULL OUTER JOIN

FULL OUTER JOIN = keep ALL rows from BOTH tables. Nothing is removed. No match = write NULL on that side.

```
SELECT e.first_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d
ON e.dept_id = d.dept_id;
```

What happens:

- Steven (90) matches Executive -----> kept
- Neena (90) matches Executive -----> kept
- Lex (NULL) -----> no match -----> **KEPT with NULL dept_name**
- Diana (10) matches Administration -----> kept
- Kimberly (10) matches Administration -----> kept
- Marketing (20) -----> no employee -----> **KEPT with NULL first_name**
- Accounting (110) -----> no employee -----> **KEPT with NULL first_name**

FULL OUTER JOIN Final Result:

first_name	dept_name
Steven	Executive
Neena	Executive
Lex	NULL
Diana	Administration
Kimberely	Administration
NULL	Marketing
NULL	Accounting

7 rows! Everyone is here. No row is removed. Lex, Marketing, and Accounting have NULL where there is no match.

Part 6: All 4 JOINS Compared

Row	INNER	LEFT	RIGHT	FULL OUTER
Steven Executive	YES	YES	YES	YES
Neena Executive	YES	YES	YES	YES
Lex NULL	NO	YES	NO	YES
Diana Admin	YES	YES	YES	YES
Kimberely Admin	YES	YES	YES	YES
NULL Marketing	NO	NO	YES	YES
NULL Accounting	NO	NO	YES	YES
Total rows	4	5	6	7

Simple rules:

JOIN Type	Rule
INNER JOIN	Only matching rows. Others are removed.
LEFT JOIN	ALL left table rows kept. Right: only matches.
RIGHT JOIN	ALL right table rows kept. Left: only matches.
FULL OUTER	ALL rows from BOTH tables. Nothing removed.

How to remember (easy trick):

LEFT = protect the LEFT table (the FROM table).
RIGHT = protect the RIGHT table (the JOIN table).
INNER = protect nobody. Only matches survive.
FULL = protect everybody. Nothing is lost.

When to use which JOIN?

You want ...	Use ...
Only employees with a department	INNER JOIN
ALL employees, even without department	LEFT JOIN
ALL departments, even without employees	RIGHT JOIN
ALL employees AND ALL departments	FULL OUTER JOIN
Find employees with no department	LEFT JOIN + WHERE dept_id IS NULL